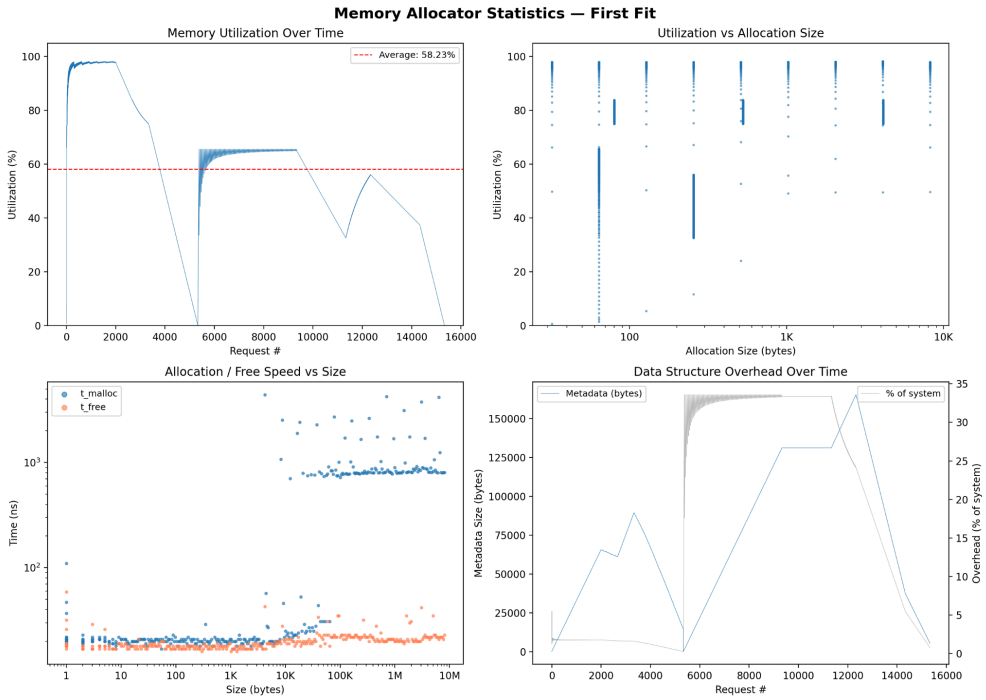
Report: Programming Assignment 6 – Memory Malloc

By: Priscilla Ye

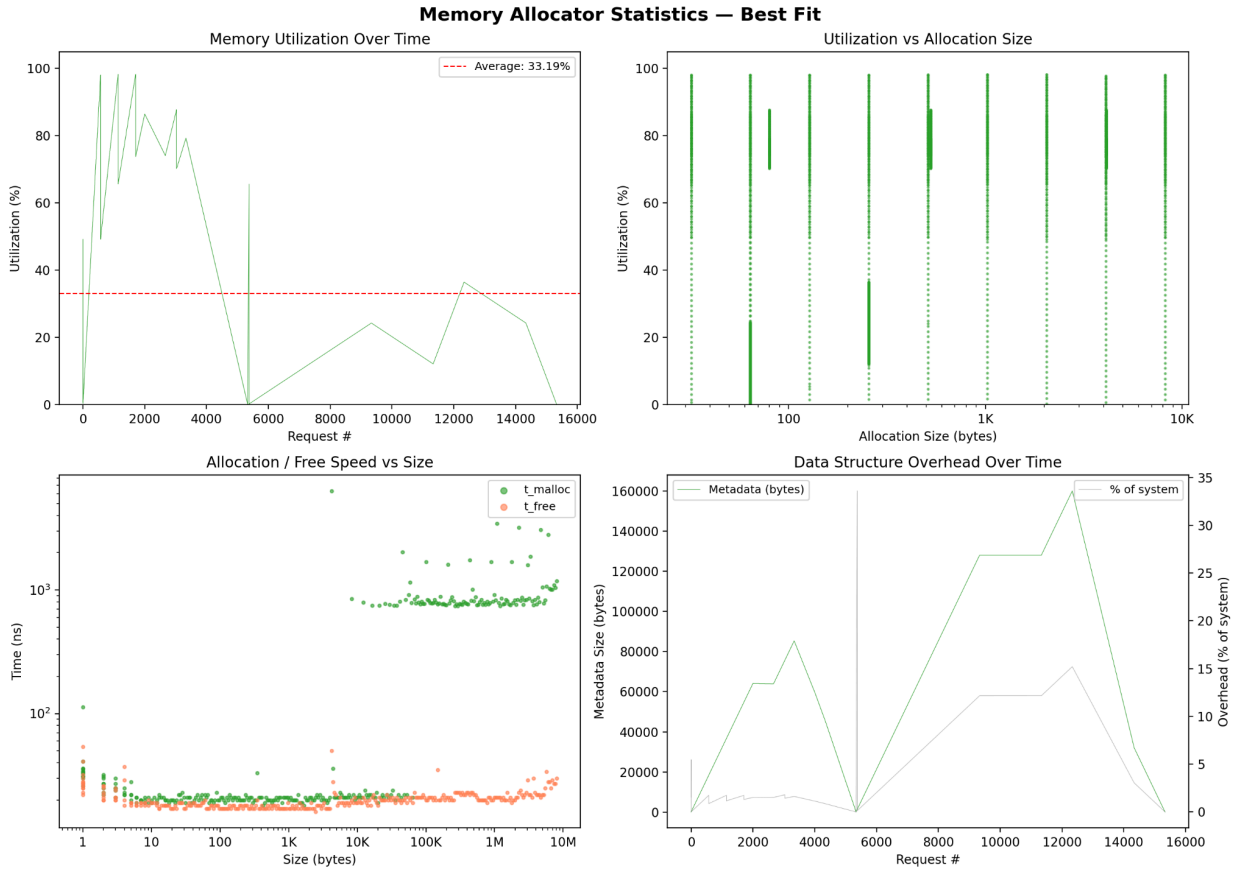
This project implements a custom dynamic memory allocator in C using mmap that supports three allocation strategies: first fit, best fit, and worst fit. The allocator manages a linked list of memory blocks with splitting and coalescing to reduce fragmentation. A test harness benchmarks each strategy across speed, utilization, and overhead metrics using realistic workload patterns including fragmentation stress tests. A Python script then visualizes the results, generating per-strategy and cross-strategy comparison charts for analysis.

Graphical Statistics Gathered: First Fit



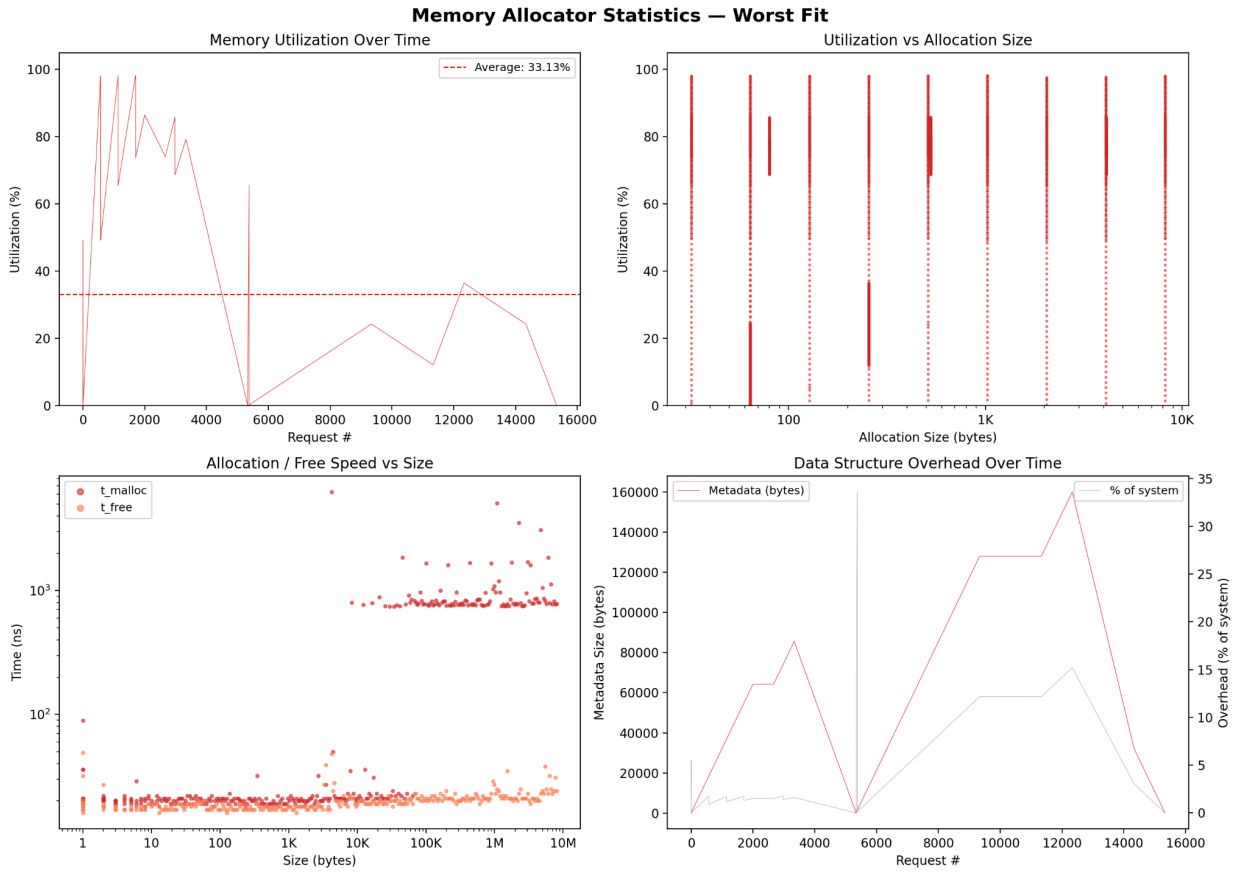
First Fit achieves the highest average utilization at 58.2%, maintaining near-peak capacity (65%) for an extended period before declining. However, this efficiency comes at a cost: metadata overhead grows substantially to 170KB at its peak, representing over 30% of system memory, indicating severe fragmentation with many small free blocks requiring tracking.

Graphical Statistics Gathered: Best Fit

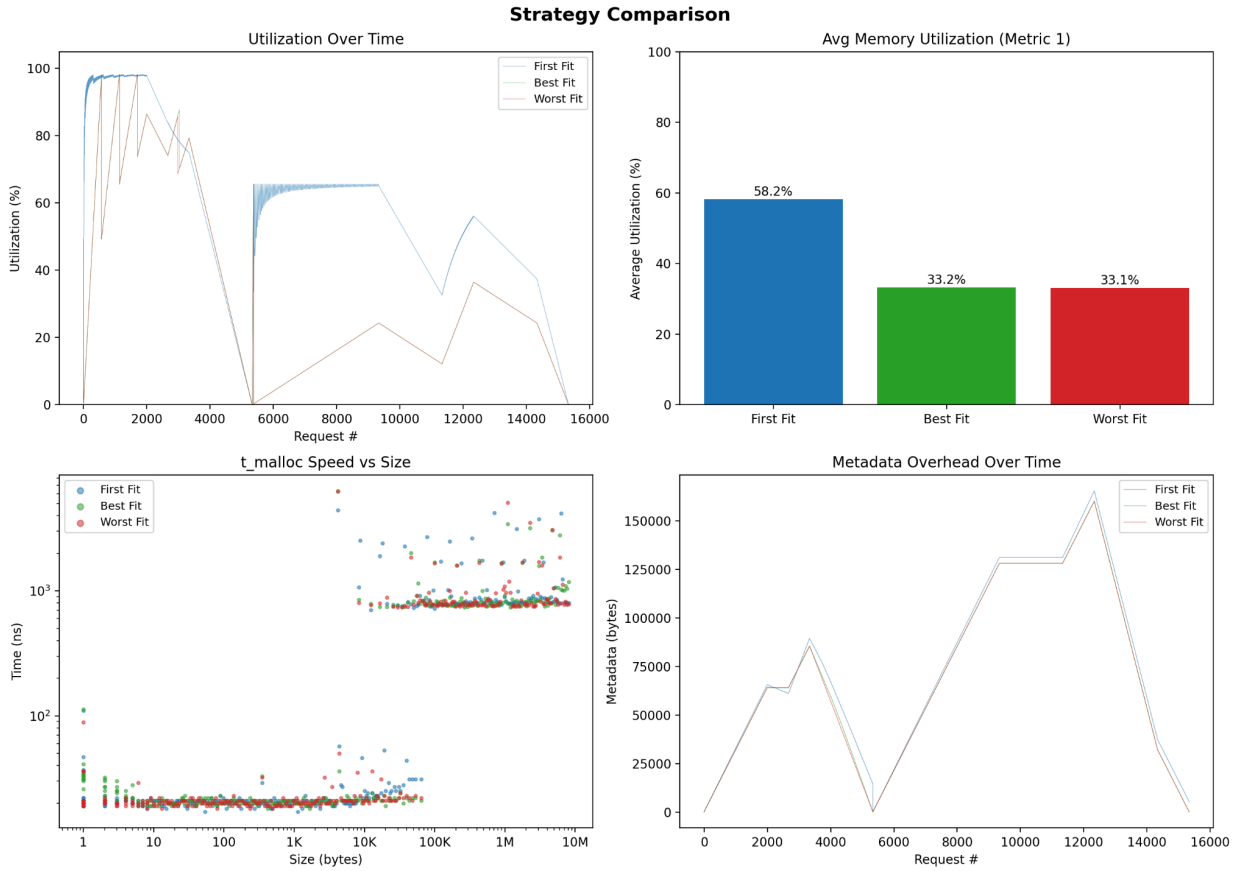


Best Fit shows poor average utilization of just 33.2% despite reaching similar peak values as First Fit. The strategy exhibits extreme volatility in the utilization pattern and highly variable allocation speeds across different request sizes. Its metadata overhead peaks around 160KB, suggesting it creates fragmented memory patterns similar to First Fit but fails to maintain sustained high utilization.

Graphical Statistics Gathered: Worst Fit



Worst Fit performs nearly identically to Best Fit with 33.1% average utilization and matching metadata overhead characteristics. The allocation/free speeds are consistent with Best Fit, and the utilization pattern shows the same volatile behavior. This similarity suggests that for this particular workload, both strategies fragment memory comparably and struggle to maintain efficient space usage.



Overall Summary

First Fit emerges as the clear winner for this workload, achieving 75% higher average utilization (58.2% vs ~33%) compared to Best and Worst Fit strategies. While all three strategies reach similar peak utilization near 98%, First Fit sustains high utilization much longer, though at the cost of elevated metadata overhead during peak usage. Best and Worst Fit perform nearly identically, both showing poor sustained utilization and high fragmentation, suggesting the allocation pattern doesn't benefit from size-based placement strategies. The dramatic performance difference highlights that for this particular mix of allocation sizes and patterns, simple First Fit placement provides significantly better memory efficiency than more sophisticated approaches.

Output:

```
=== Per-Strategy Analysis ===
[First Fit]
   Average utilization : 58.23%
   Peak utilization    : 98.18%
   Final metadata bytes: 5344
   Data points         : 15334

[Best Fit]
   Average utilization : 33.19%
   Peak utilization    : 98.25%
   Final metadata bytes: 64
   Data points         : 15334

[Worst Fit]
   Average utilization : 33.13%
   Peak utilization    : 98.18%
   Final metadata bytes: 64
   Data points         : 15334
```